Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What will be the output of the following code?

```
class Test {
    private int value;
    Test(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}
public class Main {
    public static void main(String[] args) {
        Test obj = new Test(10);
        System.out.println(obj.value);
    }
```

240701254

```
}
```

Answer

Compile-time error

Status : Correct ***Marks : 1/1***

2. What will be the output of the following code?

```
class Person {
   int age = 18;
}

public class Main {
   public static void main(String[] args) {
      Person p = new Person();
      p.age += 2;
      System.out.println("Age: " + p.age);
   }
}
```

Answer

Age: 20

Status : Correct ***Marks : 1/1***

3. What will be the output of the following code?

```
class Demo {
   void printMessage() {
      System.out.println("Hello from Demo");
   }
}

public class Main {
   public static void main(String[] args) {
      Demo d = new Demo();
      d.printMessage();
   }
```

240701254

```
}
```

Answer

Hello from Demo

Status : Correct ***Marks : 1/1***

4. What will be the output of the following code?

```
class Box {
   int length = 5;
   int width = 4;

   int area() {
      return length * width;
   }

   public static void main(String[] args) {
      Box b = new Box();
      System.out.println("Area = " + b.area());
   }
}
```

Answer

Area = 20

Status : Correct ***Marks : 1/1***

5. What will be the output of the following code?

```
class MathUtils {
   int add(int x) {
      return x + x;
   }
}

public class Main {
   public static void main(String[] args) {
      MathUtils m = new MathUtils();
```

240701254

```
        System.out.println(m.add(5));
    }
}
```

Answer

10

Status : Correct ***Marks : 1/1***

6. What will be the output of the following code?

```
class A {
    int val = 20;
}

public class Main {
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = obj1;
        obj2.val += 5;
        System.out.println(obj1.val);
    }
}
```

Answer

25

Status : Correct ***Marks : 1/1***

7. What is the output of the following code?

```
class Box {
    int height;
    Box(int height) {
        this.height = height;
    }
    void modifyHeight(Box b) {
        b.height += 10;
    }
```

240701254

```
}
public class Main {
    public static void main(String[] args) {
        Box b1 = new Box(20);
        b1.modifyHeight(b1);
        System.out.println(b1.height);
    }
}
```

Answer

30

Status : Correct ***Marks : 1/1***

8. What will be the output of the following code?

```
class Alpha {
    void greet(String name) {
        System.out.println("Hello " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Alpha obj = new Alpha();
        obj.greet("Anu");
    }
}
```

Answer

Hello Anu

Status : Correct ***Marks : 1/1***

9. What will be the output of the following code?

```
class Box {
    int volume(int l, int b, int h) {
        return l * b * h;
```

240701254

```
    }
}

public class Main {
    public static void main(String[] args) {
        Box b = new Box();
        System.out.println(b.volume(2, 3, 4));
    }
}
```

Answer

24

Status : Correct ***Marks : 1/1***

10. What will be the output of the following code?

```
class A {
    int x = 50;
}

public class Main {
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = obj1;
        obj2.x = 100;
        System.out.println(obj1.x);
    }
}
```

Answer

100

Status : Correct ***Marks : 1/1***

11. What will be the output of the following code?

```
class A {
    int p = 5;
```

```
    int q = 2;
}

class Main {
    public static void main(String[] args) {
        A obj = new A();
        System.out.println(obj.p + obj.q);
    }
}
```

Answer

7

Status : Correct ***Marks : 1/1***

12. What will be the output of the following code?

```
class A {
    int y = 30;
}

public class Main {
    public static void main(String[] args) {
        A a1 = new A();
        A a2 = new A();
        a1.y = 50;
        System.out.println(a2.y);
    }
}
```

Answer

30

Status : Correct ***Marks : 1/1***

13. What will be the output of the following code?

```
class Person {
    String name;
```

```
    void setName(String n) {
        name = n;
    }
    void printName() {
        System.out.println(name);
    }
}

class Test {
    public static void main(String[] args) {
        Person p = new Person();
        p.printName();
    }
}
```

Answer

null

Status : Correct ***Marks : 1/1***

14. What will be the output of the following code?

```
class Ball {
    int size = 11;
}

class Game {
    public static void main(String[] args) {
        Ball b1 = new Ball();
        Ball b2 = new Ball();
        b2.size = 10;
        System.out.println(b1.size);
    }
}
```

Answer

11

Status : Correct ***Marks : 1/1***

240701254

15. What will be the output of the following code?

```
class Sample {
    int x = 10;

    void display() {
        System.out.println("x = " + x);
    }

    public static void main(String[] args) {
        Sample s = new Sample();
        s.display();
    }
}
```

Answer

x = 10

Status : Correct ***Marks : 1/1***

240701254

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Elsa subscribes to a premium service with a base monthly cost, a service tax and an extra feature cost. Assist her in writing an inheritance program that takes input for these values and calculates the total monthly cost.

Refer to the below class diagram:

Input Format

The first line of input consists of a double value, representing the base monthly cost.

The second line consists of a double value, representing the service tax.

The third line consists of a double value, representing the extra feature cost.

Output Format

The output prints "Rs. X" where X is a double value, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10.0
2.5
5.0
Output: Rs. 17.50

Answer

```
import java.util.Scanner;

class Subscription {
   public double monthlyCost;
   public double serviceTax;
   public double extraFeatureCost;
}

class PremiumSubscription extends Subscription {
   public PremiumSubscription(double monthlyCost, double serviceTax, double
extraFeatureCost) {
      this.monthlyCost = monthlyCost;
      this.serviceTax = serviceTax;
      this.extraFeatureCost = extraFeatureCost;
   }

   public double calculateMonthlyCost() {
      double totalCost = monthlyCost + serviceTax + extraFeatureCost;
      return totalCost;
   }
}

public class Main {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);
```

```
        double baseMonthlyCost = scanner.nextDouble();
        double serviceTax = scanner.nextDouble();
        double extraFeatureCost = scanner.nextDouble();

        PremiumSubscription premiumSubscription = new
PremiumSubscription(baseMonthlyCost, serviceTax, extraFeatureCost);

        double totalMonthlyCost = premiumSubscription.calculateMonthlyCost();

        System.out.printf("Rs. %.2f%n", totalMonthlyCost);

        scanner.close();
    }
}
```

Status : Correct ***Marks : 10/10***

240701254

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

You are working as a developer for CityBank, which wants to build a basic account management system.

Each customer at the bank has:

An Account Number (integer)A Customer Name (string)An Initial Balance (double)

The bank allows two types of transactions:

Deposit – increases the balance.Withdrawal – decreases the balance only if enough funds are available.

If the withdrawal amount is greater than the balance, the withdrawal should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for account details.A constructor to initialize account details.Setter methods to update details if needed.Getter methods to retrieve details.Objects of the class to represent customers.

Finally, display each customer's account details after all transactions.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

Output Format

For each customer, print the details in the following format:

1. Account Number: <account_number>
2. Customer Name: <customer_name>
3. Final Balance: <final_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1234
Rahul Sharma
5000
2000
3000
Output: Account Number: 1234
Customer Name: Rahul Sharma

240701254

Final Balance: 4000.0

Answer

```
import java.util.Scanner;

class BankAccount

{


    private int accountNumber;
    private String customerName;
    private double balance;

    public BankAccount(int accountNumber, String customerName, double
balance)

{


        this.accountNumber = accountNumber;
        this.customerName = customerName;
        this.balance = balance;


}

    // Getters
    public int getAccountNumber()

{

 return accountNumber;

}
    public String getCustomerName()

{

 return customerName;
```

240701254

```
}
    public double getBalance()
{

 return balance;

}

    // Setters
    public void setAccountNumber(int accountNumber)

{

 this.accountNumber = accountNumber;

}
    public void setCustomerName(String customerName)

{

 this.customerName = customerName;

}
    public void setBalance(double balance)

{

 this.balance = balance;

}

    // Deposit method
    public void deposit(double amount)

{


        if (amount > 0)

{
```

240701254

```
        balance += amount;

    }

    }

    // Withdrawal method
    public void withdraw(double amount)

    {

        if (amount <= balance)

    {

        balance -= amount;

    }

    }
    }
    public class Main

    {

    public static void main(String[] args)

    {

        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());
```

```
    BankAccount[] accounts = new BankAccount[N];

    for (int i = 0; i < N; i++)

{


        int accNumber = Integer.parseInt(sc.nextLine());
        String name = sc.nextLine();
        double initialBalance = Double.parseDouble(sc.nextLine());
        double depositAmount = Double.parseDouble(sc.nextLine());
        double withdrawalAmount = Double.parseDouble(sc.nextLine());

        accounts[i] = new BankAccount(accNumber, name, initialBalance);
        accounts[i].deposit(depositAmount);
        accounts[i].withdraw(withdrawalAmount);


}

    for (BankAccount acc : accounts)

{


        System.out.println("Account Number: " + acc.getAccountNumber());
        System.out.println("Customer Name: " + acc.getCustomerName());
        System.out.println("Final Balance: " + Math.round(acc.getBalance() *
10.0) / 10.0);


}

    sc.close();


}

}
```

Status : Correct ***Marks : 10/10***

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

You are working as a developer for CityCab, a taxi service company that wants to build a ride fare management system.

Each customer booking has:

A Booking ID (integer)A Customer Name (string)A Distance Travelled in km (double)

The fare calculation rules are:

Base Fare = 50 units (flat charge for every ride).Per km charge = 10 units/km.If the distance is greater than 20 km, a 10% discount is applied on the total fare.

You are required to implement this system using:

A class with attributes for booking details.A constructor to initialize booking details.Setter methods to update details if needed.Getter methods to retrieve details.Objects of the class to represent customer rides.

Finally, display each booking's details and final fare.

Input Format

The first line of input contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the booking ID (integer).
- The following line contains the customer's name (string).
- The next line contains the distance travelled (double).

Output Format

For each booking, print the details in the following format:

1. Booking ID: <booking_id>
2. Customer Name: <customer_name>
3. Final Fare: <final_fare> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1234
Rahul Sharma
15

Output: Booking ID: 1234
Customer Name: Rahul Sharma
Final Fare: 200.0

Answer

```
// You are using Java
import java.util.Scanner;
```

240701254

```
class CabBooking
{

    private int bookingID;
    private String customerName;
    private double distance;

    public CabBooking(int bookingID, String customerName, double distance)

{

        this.bookingID = bookingID;
        this.customerName = customerName;
        this.distance = distance;


}

    // Getters
    public int getBookingID()

{

 return bookingID;

}
    public String getCustomerName()

{

 return customerName;

}
    public double getDistance()

{

 return distance;
```

240701254

```
}
    // Setters
    public void setBookingID(int bookingID)

{

 this.bookingID = bookingID;

}
    public void setCustomerName(String customerName)

{

 this.customerName = customerName;

}
    public void setDistance(double distance)

{

 this.distance = distance;

}

    // Method to calculate fare
    public double calculateFare()

{


        double fare = 50 + distance * 10; // Base fare + per km charge
        if (distance > 20)

{


            fare *= 0.9; // 10% discount


}
        return fare;
```

```
}

}

public class Main

{


    public static void main(String[] args)

{


        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());
        CabBooking[] bookings = new CabBooking[N];

        for (int i = 0; i < N; i++)

{


            int id = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            double distance = Double.parseDouble(sc.nextLine());

            bookings[i] = new CabBooking(id, name, distance);


}

        for (CabBooking booking : bookings)

{


            System.out.println("Booking ID: " + booking.getBookingID());
            System.out.println("Customer Name: " + booking.getCustomerName());
            System.out.println("Final Fare: " + Math.round(booking.calculateFare() *
```

```
10.0) / 10.0);

}

        sc.close();


}

}
```

Status : Correct ***Marks : 10/10***

240701254

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer)A Customer Name (string)Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units 4 per unitFor the next 100 units (51–150) 6 per unitFor units above 150 8 per unitIf the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details.A constructor to initialize customer details.Setter methods to update details if needed.Getter methods to retrieve details.Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1001
Ravi Kumar
30

Output: Customer ID: 1001
Customer Name: Ravi Kumar
Final Bill: 120.0

Answer

// You are using Java

240701254

```
import java.util.Scanner;

class Customer

{


   private int customerId;
   private String customerName;
   private double unitsConsumed;

   public Customer(int customerId, String customerName, double
unitsConsumed)

{


      this.customerId = customerId;
      this.customerName = customerName;
      this.unitsConsumed = unitsConsumed;


}

   public void setCustomerId(int customerId)

{

 this.customerId = customerId;

}
   public void setCustomerName(String customerName)

{

 this.customerName = customerName;

}
   public void setUnitsConsumed(double unitsConsumed)

{
```

240701254

```
 this.unitsConsumed = unitsConsumed;
}
   public int getCustomerId()
{
 return customerId;
}
   public String getCustomerName()
{
 return customerName;
}
   public double getUnitsConsumed()
{
 return unitsConsumed;
}
   public double calculateBill()
{

      double bill = 0;
      double remaining = unitsConsumed;

      if (remaining > 0)
{

         double first50 = Math.min(50, remaining);
         bill += first50 * 4;
         remaining -= first50;
```

240701254

```
}
        if (remaining > 0)
{

            double next100 = Math.min(100, remaining);
            bill += next100 * 6;
            remaining -= next100;

}
        if (remaining > 0)
{

            bill += remaining * 8;

}
        if (bill > 2000) bill -= bill * 0.15;
        return bill;

}
}
public class Main
{

    public static void main(String[] args)
```

240701254

```
{

    Scanner sc = new Scanner(System.in);
    int N = Integer.parseInt(sc.nextLine());

    for (int i = 0; i < N; i++)

{


        int id = Integer.parseInt(sc.nextLine());
        String name = sc.nextLine();
        double units = Double.parseDouble(sc.nextLine());

        Customer customer = new Customer(id, name, units);

        System.out.println("Customer ID: " + customer.getCustomerId());
        System.out.println("Customer Name: " + customer.getCustomerName());
        System.out.printf("Final Bill: %.1f\n", customer.calculateBill());


}
    sc.close();


}
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household’s water account has:

A Customer ID (integer)A Customer Name (string)Liters Consumed

(double)

The water bill is calculated based on these rules:

For the first 500 liters 2 per literFor the next 500 liters (501–1000) 3 per literFor liters above 1000 5 per literIf the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details.A constructor to initialize customer details.Setter methods to update details if needed.Getter methods to retrieve details.Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

240701254

1001
Ravi Kumar
300

Output: Customer ID: 1001
Customer Name: Ravi Kumar
Final Bill: 600.0

Answer

```
// You are using Java
import java.util.Scanner;

class Customer

{


    private int customerId;
    private String customerName;
    private double litersConsumed;

    public Customer(int customerId, String customerName, double
litersConsumed)

{


        this.customerId = customerId;
        this.customerName = customerName;
        this.litersConsumed = litersConsumed;


}

    public void setCustomerId(int customerId)

{

 this.customerId = customerId;

}
    public void setCustomerName(String customerName)
```

240701254

```
{
 this.customerName = customerName;
}
   public void setLitersConsumed(double litersConsumed)
{
 this.litersConsumed = litersConsumed;
}
   public int getCustomerId()
{
 return customerId;
}
   public String getCustomerName()
{
 return customerName;
}
   public double getLitersConsumed()
{
 return litersConsumed;
}
   public double calculateBill()
{
      double bill = 0;
      double remaining = litersConsumed;
```

240701254

```
    if (remaining > 0)
{

        double first500 = Math.min(500, remaining);
        bill += first500 * 2;
        remaining -= first500;

}

    if (remaining > 0)
{

        double next500 = Math.min(500, remaining);
        bill += next500 * 3;
        remaining -= next500;

}

    if (remaining > 0)
{

        bill += remaining * 5;

}

    if (bill > 3000) bill -= bill * 0.10;

    return bill;

}
```

240701254

```
}
public class Main
{

    public static void main(String[] args)
{

        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());

        for (int i = 0; i < N; i++)
{

            int id = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            double liters = Double.parseDouble(sc.nextLine());

            Customer customer = new Customer(id, name, liters);

            System.out.println("Customer ID: " + customer.getCustomerId());
            System.out.println("Customer Name: " + customer.getCustomerName());
            System.out.printf("Final Bill: %.1f\n", customer.calculateBill());


}
        sc.close();


}

}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are working as a developer for CityMobile, which wants to build a basic mobile data usage management system.

Each customer has:

A Customer ID (integer)A Customer Name (string)An Initial Data Balance (in GB, double)

The company allows two types of operations:

Recharge – increases the data balance.Usage – decreases the data balance only if enough data is available.

If the usage amount is greater than the available data balance, the usage should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for customer details.A constructor to initialize customer details.Setter methods to update details if needed.Getter methods to retrieve details.Objects of the class to represent customers.

Finally, display each customer's details after all operations.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Initial Data Balance (double).
- The next line contains the Recharge Amount in GB (double).
- The next line contains the Usage Amount in GB (double).

Output Format

For each customer, print the details in the following format:

240701254

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Data Balance: <final_data_balance> GB (The final balance must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1234
Ravi Kumar
5.0
2.0
3.0

Output: Customer ID: 1234
Customer Name: Ravi Kumar
Final Data Balance: 4.0 GB

Answer

```
// You are using Java
import java.util.Scanner;

class Customer

{


    private int customerId;
    private String customerName;
    private double dataBalance;

    public Customer(int customerId, String customerName, double dataBalance)

{


        this.customerId = customerId;
```

240701254

```
        this.customerName = customerName;
        this.dataBalance = dataBalance;

    }

      public void setCustomerId(int customerId)

    {

     this.customerId = customerId;

    }
      public void setCustomerName(String customerName)

    {

     this.customerName = customerName;

    }
      public void setDataBalance(double dataBalance)

    {

     this.dataBalance = dataBalance;

    }

      public int getCustomerId()

    {

     return customerId;

    }
      public String getCustomerName()

    {

     return customerName;

    }
```

240701254

```
    public double getDataBalance()
{
 return dataBalance;
}
    public void recharge(double amount)
{

        dataBalance += amount;

}
    public void useData(double amount)
{

        if (amount <= dataBalance)
{
            dataBalance -= amount;

}

}
}
public class Main
{
```

240701254

```
public static void main(String[] args)
{

    Scanner sc = new Scanner(System.in);
    int N = Integer.parseInt(sc.nextLine());

    for (int i = 0; i < N; i++)
{

        int id = Integer.parseInt(sc.nextLine());
        String name = sc.nextLine();
        double initialBalance = Double.parseDouble(sc.nextLine());
        double rechargeAmount = Double.parseDouble(sc.nextLine());
        double usageAmount = Double.parseDouble(sc.nextLine());

        Customer customer = new Customer(id, name, initialBalance);
        customer.recharge(rechargeAmount);
        customer.useData(usageAmount);

        System.out.println("Customer ID: " + customer.getCustomerId());
        System.out.println("Customer Name: " + customer.getCustomerName());
        System.out.printf("Final Data Balance: %.1f GB\n",
customer.getDataBalance());


}
    sc.close();


}

}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Anjali is now working as a developer for the City Marathon Association, which wants to build a system to track and find the fastest runner among marathon participants.

Each runner's record has:

Runner ID (integer)Runner Name (string)An array of times (in minutes) taken in 5 marathon events (integers)

The system must calculate:

The average time of each runner (sum of all times / 5).Identify the fastest runner (the one with the lowest average time).If two or more runners have the same average time, the one with the lower Runner ID is considered the fastest runner.

Anjali has been asked to implement this system using:

A class with attributes for runner details.A constructor to initialize runner details.Getter and Setter methods to retrieve and update runner details if required.A method to calculate the average time.Objects of the class to represent runners.

Finally, display each runner's details and announce the Fastest Runner.

Input Format

The first line of input contains an integer N (number of runners).

For each runner:

- The next line contains the Runner ID (integer).
- The following line contains the Runner Name (string).
- The next line contains 5 integers separated by spaces (times in minutes for 5 marathon events).

Output Format

For each runner the output prints the following details:

- Runner ID: <runner_id>
- Runner Name: <runner_name>
- Average Time: <average_time>

240701254

Finally, print "Fastest Runner: <runner_name> with <average_time> minutes"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1001
Ravi Kumar
240 250 245 255 260
Output: Runner ID: 1001
Runner Name: Ravi Kumar
Average Time: 250
Fastest Runner: Ravi Kumar with 250 minutes

Answer

```
// You are using Java
import java.util.Scanner;

class Runner

{


    private int runnerId;
    private String runnerName;
    private int[] times;
    private int averageTime;

    public Runner(int runnerId, String runnerName, int[] times)

{


        this.runnerId = runnerId;
        this.runnerName = runnerName;
```

```
        this.times = times;
        calculateAverageTime();

    }

      public int getRunnerId()

    {

     return runnerId;

    }
      public String getRunnerName()
    {

     return runnerName;

    }
      public int getAverageTime()

    {

     return averageTime;

    }

      public void setRunnerId(int runnerId)

    {

     this.runnerId = runnerId;

    }
      public void setRunnerName(String runnerName)

    {

     this.runnerName = runnerName;
    }
```

240701254

```
    public void setTimes(int[] times)
{
 this.times = times; calculateAverageTime();
}
    public void calculateAverageTime()
{

        int sum = 0;
        for (int t : times) sum += t;
        averageTime = sum / times.length;

}
}
public class Main
{

    public static void main(String[] args)
{

        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());
        Runner[] runners = new Runner[N];

        for (int i = 0; i < N; i++)
{

            int id = Integer.parseInt(sc.nextLine());
```

```
            String name = sc.nextLine();
            String[] timeStr = sc.nextLine().split(" ");
            int[] times = new int[5];
            for (int j = 0; j < 5; j++) times[j] = Integer.parseInt(timeStr[j]);
            runners[i] = new Runner(id, name, times);


    }

        Runner fastest = runners[0];

        for (Runner r : runners)

    {


            System.out.println("Runner ID: " + r.getRunnerId());
            System.out.println("Runner Name: " + r.getRunnerName());
            System.out.println("Average Time: " + r.getAverageTime());
            if (r.getAverageTime() < fastest.getAverageTime() ||
                (r.getAverageTime() == fastest.getAverageTime() && r.getRunnerId() <
fastest.getRunnerId()))

    {


                fastest = r;


    }


    }

        System.out.println("Fastest Runner: " + fastest.getRunnerName() + " with " +
fastest.getAverageTime() + " minutes");
        sc.close();


    }
}
```

Status : Correct

Marks : 10/10

240701254

Rajalakshmi Engineering College

Name: Keerthana G
Email: 240701254@rajalakshmi.edu.in
Roll no: 240701254
Phone: 9342478755
Branch: REC
Department: CSE - Section 3
Batch: 2028
Degree: B.E - CSE

Scan to verify results

2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Ravi is working as a developer for SecureLogin Systems, which wants to build a system to evaluate the strength of user passwords.

Each user record has:

User ID (integer)User Name (string)Password (string)

The system must calculate whether a password is strong or weak.

A password is considered strong if it meets all of the following conditions:

At least 8 characters long.Contains at least one uppercase letter.Contains at least one lowercase letter.Contains at least one digit.Contains at least one special character (from !@#$%^&*).

Ravi has been asked to implement this system using:

A class with attributes for user details.A constructor to initialize user details.Getter and setter methods to retrieve or update user details.A method to check whether the password is strong.Objects of the class to represent users.

Finally, display each user's details and indicate whether their password is Strong or Weak.

Input Format

The first line contains an integer N, representing the number of users.

For each user:

The next line contains the User ID (integer).

The next line contains the User Name (string).

The next line contains the Password (string).

Output Format

For each user, print the details in the following format:

User ID: <user_id>

User Name: <user_name>

Password: <password>

Password Strength: <Strong/Weak>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1001
Ravi Kumar

240701254

Abc@1234

Output: User ID: 1001
User Name: Ravi Kumar
Password: Abc@1234
Password Strength: Strong

Answer

```
// You are using Java
import java.util.Scanner;

class User

{


    private int userId;
    private String userName;
    private String password;

    public User(int userId, String userName, String password)

{


        this.userId = userId;
        this.userName = userName;
        this.password = password;


}

    public int getUserId()

{

 return userId;

}
    public String getUserName()

{
```

240701254

```
 return userName;
}
   public String getPassword()
{
 return password;
}
   public void setUserId(int userId)
{
 this.userId = userId;
}
   public void setUserName(String userName)
{
 this.userName = userName;
}
   public void setPassword(String password)
{
 this.password = password;
}
   public String checkPasswordStrength()
{

      if (password.length() < 8) return "Weak";
      boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial =
false;
```

```
        for (char c : password.toCharArray())
{

            if (Character.isUpperCase(c)) hasUpper = true;
            else if (Character.isLowerCase(c)) hasLower = true;
            else if (Character.isDigit(c)) hasDigit = true;
            else if ("!@#$%^&*".indexOf(c) >= 0) hasSpecial = true;


}

        return (hasUpper && hasLower && hasDigit && hasSpecial) ? "Strong" :
"Weak";


}

}

public class Main

{


    public static void main(String[] args)
{


        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());
        User[] users = new User[N];

        for (int i = 0; i < N; i++)

{


            int id = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
```

```
            String pass = sc.nextLine();
            users[i] = new User(id, name, pass);


    }

        for (User u : users)

    {


            System.out.println("User ID: " + u.getUserId());
            System.out.println("User Name: " + u.getUserName());
            System.out.println("Password: " + u.getPassword());
            System.out.println("Password Strength: " + u.checkPasswordStrength());


    }

        sc.close();


    }

    }
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Each customer at the bank has an Account Number, Customer Name, and an Initial Balance. The bank allows two types of transactions:

Deposit – Increases the balance.Withdrawal – Decreases the balance, but only if enough funds are available. If the withdrawal amount exceeds the available balance, the transaction should be skipped, and the balance should remain unchanged.

You are required to implement this banking system by:

Creating a class with the necessary attributes to store account details.

Using a constructor to initialize the account details when a new account is created.Providing setter methods to update the details if required.Providing getter methods to retrieve account details.Creating objects of this class to represent different customers, where each customer can perform deposits and withdrawals.

Instructions:

Implement the class to store account details.Implement the logic for performing deposit and withdrawal transactions.Ensure that withdrawals don't exceed the available balance.After performing the transactions, print the account number, customer name, and final balance.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

Output Format

For each customer, print the details in the following format:

1. Account Number: <account_number>
2. Customer Name: <customer_name>
3. Final Balance: <final_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1234
Rahul Sharma
5000

240701254

2000
3000
Output: Account Number: 1234
Customer Name: Rahul Sharma
Final Balance: 4000.0

Answer

```
// You are using Java
import java.util.Scanner;

class BankAccount

{


   private int accountNumber;
   private String customerName;
   private double balance;

   public BankAccount(int accountNumber, String customerName, double
balance)

{


      this.accountNumber = accountNumber;
      this.customerName = customerName;
      this.balance = balance;


}

   public int getAccountNumber()

{

 return accountNumber;

}
   public String getCustomerName()
{
```

240701254

```
 return customerName;
}
   public double getBalance()
{
 return balance;
}
   public void setAccountNumber(int accountNumber)
{
 this.accountNumber = accountNumber;
}
   public void setCustomerName(String customerName)
{
 this.customerName = customerName;
}
   public void setBalance(double balance)
{
 this.balance = balance;
}
   public void deposit(double amount)
{

    balance += amount;
```

```
}
   public void withdraw(double amount)
{

      if (amount <= balance)
{

         balance -= amount;
}

}
}
public class Main
{

   public static void main(String[] args)
{

      Scanner sc = new Scanner(System.in);
      int N = Integer.parseInt(sc.nextLine());
      BankAccount[] accounts = new BankAccount[N];

      for (int i = 0; i < N; i++)
{

         int accNo = Integer.parseInt(sc.nextLine());
```

```
        String name = sc.nextLine();
        double initialBalance = Double.parseDouble(sc.nextLine());
        double depositAmount = Double.parseDouble(sc.nextLine());
        double withdrawAmount = Double.parseDouble(sc.nextLine());

        BankAccount acc = new BankAccount(accNo, name, initialBalance);
        acc.deposit(depositAmount);
        acc.withdraw(withdrawAmount);
        accounts[i] = acc;


}

    for (BankAccount acc : accounts)

{


        System.out.println("Account Number: " + acc.getAccountNumber());
        System.out.println("Customer Name: " + acc.getCustomerName());
        System.out.println("Final Balance: " + String.format("%.1f",
acc.getBalance()));


}

    sc.close();


}

}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Neha is working as a developer for CityMovie Theatre, which wants to build a system to calculate total ticket cost for movie-goers based on the number of tickets and type of seats booked.

Each customer’s booking has:

Booking ID (integer)Customer Name (string)Number of Tickets (integer)Seat Type (string: "Standard", "Premium", "VIP")

The ticket prices are:

Standard – 250 units per ticketPremium – 400 units per ticketVIP – 600 units per ticket

The calculation rules:

Total Amount = Number of Tickets × Seat Price

If a customer books more than 4 tickets, they get a 10% discount on the total amount.

If the booking is for VIP seats and the total amount exceeds 3000 units, a 5% luxury tax is added after any discount.

Neha has been asked to implement this system using:

A class with attributes for booking details.A constructor to initialize booking details.Getter and Setter methods to retrieve and update booking details if required.A method to calculate the final ticket cost.Objects of the class to represent bookings.

Finally, display each customer’s details and final ticket amount.

Input Format

The first line contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the Booking ID (integer).
- The next line contains the Customer Name (string).
- The next line contains Number of Tickets (integer).
- The next line contains Seat Type ("Standard", "Premium", or "VIP").

Output Format

For each booking, print:

240701254

- Booking ID: <booking_id>
- Customer Name: <customer_name>
- Final Ticket Amount: <final_amount> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1001
Ravi Kumar
3
Standard
Output: Booking ID: 1001
Customer Name: Ravi Kumar
Final Ticket Amount: 750.0

Answer

```
// You are using Java
import java.util.Scanner;

class Booking

{


    private int bookingID;
    private String customerName;
    private int numTickets;
    private String seatType;

    public Booking(int bookingID, String customerName, int numTickets, String
seatType)

{


        this.bookingID = bookingID;
        this.customerName = customerName;
```

```
        this.numTickets = numTickets;
        this.seatType = seatType;

    }

      public int getBookingID()

    {

     return bookingID;

    }
      public String getCustomerName()

    {

     return customerName;

    }
      public int getNumTickets()

    {

     return numTickets;

    }
      public String getSeatType()

    {

     return seatType;

    }

      public double calculateFinalAmount()

    {

        double seatPrice = 0;
        switch (seatType)
```

```
{

        case "Standard": seatPrice = 250; break;
        case "Premium": seatPrice = 400; break;
        case "VIP": seatPrice = 600; break;


}

    double totalAmount = numTickets * seatPrice;

    // Apply 10% discount for more than 4 tickets
    if (numTickets > 4)
{


        totalAmount *= 0.9;


}

    // Apply 5% luxury tax for VIP if total > 3000
    if (seatType.equals("VIP") && totalAmount > 3000)
{


        totalAmount *= 1.05;


}

    return totalAmount;


}

}
```

240701254

```
public class Main
{

    public static void main(String[] args)
{

        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());
        Booking[] bookings = new Booking[N];

        for (int i = 0; i < N; i++)
{

            int bookingID = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            int tickets = Integer.parseInt(sc.nextLine());
            String seat = sc.nextLine();

            bookings[i] = new Booking(bookingID, name, tickets, seat);

}

        for (Booking b : bookings)
{

            System.out.println("Booking ID: " + b.getBookingID());
            System.out.println("Customer Name: " + b.getCustomerName());
            System.out.println("Final Ticket Amount: " + String.format("%.1f",
b.calculateFinalAmount()));

}
```

240701254

```
    sc.close();

}

}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Neha is working as a developer for CityQuiz Platform, which wants to build a system to calculate quiz scores and identify top scorers among participants.

Each participant's record has:

Participant ID (integer)Participant Name (string)An array of scores in 5 quiz rounds (integers, each between 0 and 100)

The system must calculate:

Total Score = sum of scores in all 5 rounds.Average Score = Total Score ÷ 5.If a participant scores above 80 in all rounds, a bonus of 10 points is added to the total score.Identify the Top Scorer among all participants. If two participants have the same total score, the one with the lower Participant ID is considered the top scorer.

Neha has been asked to implement this system using:

A class with attributes for participant details.A constructor to initialize participant details.Getter and setter methods to retrieve or update participant details.A method to calculate total score and average score (including bonus if applicable).Objects of the class to represent participants.

Finally, display each participant's details and announce the Top Scorer.

Input Format

The first line of input contains an integer N, representing the number of participants.

240701254

For each participant:

- Next line: Participant ID (integer)
- Next line: Participant Name (string)
- Next line: 5 integers separated by spaces (scores for 5 quiz rounds)

Output Format

For each participant:

- Participant ID: <participant_id>
- Participant Name: <participant_name>
- Total Score: <total_score>
- Average Score: <average_score>

Finally, print "Top Scorer: <participant_name> with <total_score> points"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1001
Ravi Kumar
85 90 88 92 87

Output: Participant ID: 1001
Participant Name: Ravi Kumar
Total Score: 452
Average Score: 90
Top Scorer: Ravi Kumar with 452 points

Answer

```
// You are using Java
import java.util.Scanner;

class Participant
```

240701254

```
{

    private int participantID;
    private String participantName;
    private int[] scores;
    private int totalScore;
    private int averageScore;

    public Participant(int participantID, String participantName, int[] scores)

{

        this.participantID = participantID;
        this.participantName = participantName;
        this.scores = scores;
        calculateScores();


}

    public int getParticipantID()

{

 return participantID;

}
    public String getParticipantName()

{

 return participantName;

}
    public int getTotalScore()

{

 return totalScore;
```

240701254

```
}
   public int getAverageScore()
{
 return averageScore;
}
   public void calculateScores()
{
      totalScore = 0;
      boolean bonusEligible = true;
      for (int score : scores)
{
         totalScore += score;
         if (score <= 80) bonusEligible = false;
}
      if (bonusEligible) totalScore += 10;  // add bonus
      averageScore = totalScore / scores.length;
}
}
public class Main
{
   public static void main(String[] args)
{
```

```
        Scanner sc = new Scanner(System.in);
        int N = Integer.parseInt(sc.nextLine());
        Participant[] participants = new Participant[N];

        for (int i = 0; i < N; i++)

    {


            int id = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            int[] scores = new int[5];
            String[] scoreStr = sc.nextLine().split(" ");
            for (int j = 0; j < 5; j++)

    {


                scores[j] = Integer.parseInt(scoreStr[j]);


    }
            participants[i] = new Participant(id, name, scores);


    }

        Participant topScorer = participants[0];

        for (Participant p : participants)

    {


            System.out.println("Participant ID: " + p.getParticipantID());
            System.out.println("Participant Name: " + p.getParticipantName());
            System.out.println("Total Score: " + p.getTotalScore());
            System.out.println("Average Score: " + p.getAverageScore());

            if (p.getTotalScore() > topScorer.getTotalScore() ||
```

```
        (p.getTotalScore() == topScorer.getTotalScore() && p.getParticipantID()
< topScorer.getParticipantID()))
{

            topScorer = p;

}

}
    System.out.println("Top Scorer: " + topScorer.getParticipantName() + " with "
+ topScorer.getTotalScore() + " points");
    sc.close();

}

}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Anjali is working as a developer for CityFitness Gym, which wants to build a system to calculate monthly membership fees for gym members based on the type of membership and the number of personal training sessions booked.

Each member's record has:

Member ID (integer)Member Name (string)Membership Type (string: "Basic", "Premium", "Elite")Number of Personal Training Sessions (integer)

The monthly fees are:

Basic – 1000 unitsPremium – 1500 unitsElite – 2000 units

The cost of personal training sessions is 500 units per session.

The calculation rules:

Total Amount = Membership Fee + (Number of Personal Training Sessions × 500)If the number of sessions is more than 5, a 10% discount is applied on the total amount.If the member has Elite membership and the total amount exceeds 4000, an additional 5% service tax is added after discount.

Anjali has been asked to implement this system using:

A class with attributes for member details.A constructor to initialize member details.Getter and Setter methods to retrieve and update member details if required.A method to calculate the final monthly fee.Objects of the class to represent members.

Finally, display each member's details and the final monthly fee.

Input Format

The first line contains an integer N, representing the number of members.

For each member:

- Next line contains Member ID (integer)
- Next line contains Member Name (string)
- Next line contains Membership Type ("Basic", "Premium", "Elite")
- Next line contains Number of Personal Training Sessions (integer)

Output Format

For each member, print:

- Member ID: <member_id>
- Member Name: <member_name>
- Final Monthly Fee: <final_fee> (The final fee must be rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

240701254

Input: 1
1001
Ravi Kumar
Basic
3
Output: Member ID: 1001
Member Name: Ravi Kumar
Final Monthly Fee: 2500.0

Answer

```
// You are using Java
import java.util.Scanner;

class Member
{


    private int memberID;
    private String memberName;
    private String membershipType;
    private int personalTrainingSessions;
    private double finalFee;

    public Member(int memberID, String memberName, String membershipType,
int personalTrainingSessions)
{


        this.memberID = memberID;
        this.memberName = memberName;
        this.membershipType = membershipType;
        this.personalTrainingSessions = personalTrainingSessions;
        calculateFinalFee();


}

    public int getMemberID()
{
```

```
 return memberID;
}
   public String getMemberName()
{
 return memberName;
}
   public double getFinalFee()
{
 return finalFee;
}
   public void calculateFinalFee()
{

      double membershipFee = 0;
      switch (membershipType)
{

         case "Basic": membershipFee = 1000; break;
         case "Premium": membership Fee = 1500; break;
         case "Elite": membershipFee = 2000; break;

}
      double totalAmount = membershipFee + (personalTrainingSessions * 500);

      // Apply 10% discount if sessions > 5
      if (personalTrainingSessions > 5)
```

```
{

        totalAmount *= 0.9;

}

    // Apply 5% service tax if Elite and total > 4000
    if (membershipType.equals("Elite") && totalAmount > 4000)

{

        totalAmount *= 1.05;

}

    finalFee = Math.round(totalAmount * 10.0) / 10.0;

}

}

public class Main
{

   public static void main(String[] args)

{

    Scanner sc = new Scanner(System.in);
    int N = Integer.parseInt(sc.nextLine());
    Member[] members = new Member[N];

    for (int i = 0; i < N; i++)
```

240701254

```
{

        int id = Integer.parseInt(sc.nextLine());
        String name = sc.nextLine();
        String type = sc.nextLine();
        int sessions = Integer.parseInt(sc.nextLine());
        members[i] = new Member(id, name, type, sessions);


}

    for (Member m : members)
{

        System.out.println("Member ID: " + m.getMemberID());
        System.out.println("Member Name: " + m.getMemberName());
        System.out.println("Final Monthly Fee: " + m.getFinalFee());


}

    sc.close();


}
}
```

Status : Correct ***Marks : 10/10***

240701254